

## Phase-5 Project Development

|              |                       |
|--------------|-----------------------|
| Date         | 28-09-2026            |
| Team ID      | SWTID-2026-7624       |
| Project name | Script-controlled ACL |
| Maximum mark | 4 Marks               |

### PROJECT DEVELOPMENT

The project development phase focuses on implementing the Script-Controlled Access Control List (ACL) in ServiceNow to restrict access to records based on user roles and field values. The main purpose of this project is to provide record-level security for the Institution Details table and ensure that only authorized users can view, create, modify, or delete records.

#### 5.1 User and Role Creation

The first step in project development is creating the required users and custom roles in the ServiceNow instance. A test user named EEE User is created under User Administration → Users. Custom roles such as bb1, bb2, bb3, and bb4 are also created.

These roles are used to control different levels of access to the records. The required roles are assigned to the test user so that the ACL permissions can be tested effectively.

#### 5.2 Creation of Institution Details Table

A custom table named Institution Details is created in ServiceNow.

Table Name: "u\_institution\_details"

The table contains the following fields:

- Student Roll Number – Auto Number
- Student Name – Reference to User
- Faculty Name – Reference to User
- Branch – Choice field
- Email – String

- Phone Number – String
- Description – Multi String

The Branch field contains three possible values: ECE, EEE, and CSE. Multiple records are created with different branch values so that the ACL restrictions can be tested.

### 5.3 Implementation of READ ACL

A record-level Read ACL is created for the "u\_institution\_details" table.

The ACL is configured with:

- Type: Record
- Operation: Read
- Name: "u\_institution\_details"
- Active: True
- Required Role: "bb1"
- Data Condition: Branch is EEE
- Advanced: True

The script-controlled ACL checks the user's role before allowing access. Administrators are given full access, while users with the required custom role are permitted to access the records according to the configured security condition. Users who do not satisfy the required conditions are denied access.

This ensures that unauthorized users cannot freely view protected records.

### 5.4 Testing of READ ACL

After creating the Read ACL, the access control is tested by impersonating different users.

The following conditions are verified:

- A user with the bb1 role can view the permitted EEE branch records.
- A user without the required role cannot view the records.
- An administrator can view all records regardless of the branch value.

This testing confirms that the Read ACL is controlling record visibility according to the defined access rules.

## 5.5 Implementation of CREATE ACL

A Create ACL is then developed for the same table.

The configuration includes:

- Type: Record
- Operation: Create
- Name: "u\_institution\_details"
- Active: True
- Required Role: "bb2"

The Create ACL controls whether a user is permitted to create new records in the Institution Details table. The user must have the required role in addition to the permissions necessary to access the table.

The functionality is tested using a user having the required roles, and the availability of the New button is verified.

## 5.6 Implementation of WRITE ACL

A Write ACL is created to control modification of existing records.

The configuration includes:

- Type: Record

- Operation: Write
- Name: "u\_institution\_details"
- Active: True
- Required Role: "bb3"

The Write ACL ensures that only authorized users can edit the records. Users who satisfy the required role conditions can modify the permitted records, while users without the required permission cannot edit them.

The functionality is verified using a user having bb1, bb2, and bb3 roles.

## 5.7 Implementation of DELETE ACL

The final access control implemented is the Delete ACL.

The configuration includes:

- Type: Record
- Operation: Delete
- Name: "u\_institution\_details"
- Active: True
- Required Role: "bb4"

The Delete ACL prevents unauthorized users from deleting records. Only users who satisfy the required role conditions are allowed to perform the delete operation.

The functionality is tested with a user having bb1, bb2, bb3, and bb4 roles. The user can view, create, edit, and delete the permitted records.

## 5.8 Integration of ACLs

All four ACL operations are integrated into the Institution Details table:

ACL Operation | Role | Purpose

READ | bb1 | Controls record viewing

CREATE | bb2 | Controls creation of records

WRITE | bb3 | Controls modification of records

DELETE | bb4 | Controls deletion of records

The combination of these ACLs provides multiple levels of record protection. Each operation is independently controlled through the corresponding role and ACL configuration.

## 5.9 Project Testing

The developed ACL system is tested using different users and role combinations. The system verifies whether the user can perform the required operation based on the assigned roles and configured ACL conditions.

Testing is performed for viewing, creating, editing, and deleting records. The results confirm that the access restrictions are applied according to the implemented ACL rules.

## 5.10 Project Outcome

The project successfully demonstrates the implementation of Script-Controlled ACLs in ServiceNow. The system provides record-level security by controlling access based on user roles and record field values.

The implementation covers READ, CREATE, WRITE, and DELETE operations and helps prevent unauthorized viewing, modification, creation, and deletion of sensitive records.